



Claude Code Cheatsheet

Your terminal-native pair-programmer · slash, hooks, MCP & more

CLI · v2026

Beginner → Power User

Slash · Hooks · MCP · Plugins

1 Get Started in 30 Seconds

Install & launch

```
# install (one-liner)
npm install -g @anthropic-ai/claude-code

# start in your repo
cd ~/code/myapp
claude

# first run: log in
claude /login

# health check
claude --version
claude /doctor
```

First time? Run `/init` in your repo — Claude scans the codebase and writes a starter `CLAUDE.md` for you.

Three modes you'll actually use

<code>claude</code>	interactive REPL (default)
<code>claude -p "fix bug"</code>	one-shot · prints & exits
<code>claude -c</code>	continue last session
<code>claude -r</code>	pick a session to resume
<code>claude --model opus</code>	choose model (opus/sonnet/haiku)
<code>claude mcp ...</code>	manage MCP servers (\$5)

Headless & scriptable: pipe stdin into `claude -p` to use it like any unix tool.

2 Live in the REPL

Special prefixes & keys

<code>/</code>	run a slash command (auto-completes)	<code>Shift + Tab</code>	cycle permission modes (\$6)	<code>Ctrl + L</code>	clear the screen
<code>@</code>	reference a file or folder by path	<code>Esc</code>	interrupt Claude mid-response	<code>Ctrl + C</code>	cancel input · twice to exit
<code>#</code>	save a note to memory (CLAUDE.md)	<code>Esc Esc</code>	rewind to a previous turn	<code>Ctrl + D</code>	exit on empty line
<code>!</code>	run a bash command without leaving	<code>Ctrl + R</code>	toggle verbose / tool details	<code>↑ ↓</code>	walk through prompt history

Anatomy of a slash command

`/model opus`

● slash starts a meta-command ● verb (Tab auto-completes) ● argument (varies by verb)

Tip: press `/` then `Tab` to browse every available command — built-in, custom, and from plugins.

Built-in slash commands

<code>/help</code>	show all commands
<code>/clear</code>	wipe context, fresh start
<code>/compact [hint]</code>	summarize chat to free context
<code>/init</code>	create a starter CLAUDE.md
<code>/memory</code>	edit memory files
<code>/model [name]</code>	switch model
<code>/permissions</code>	manage allow/deny rules
<code>/agents</code>	view & edit subagents
<code>/mcp</code>	connect & auth MCP servers
<code>/hooks</code>	edit lifecycle hooks
<code>/plugin</code>	install / list plugins
<code>/cost</code>	token spend so far
<code>/config · /status</code>	view settings & session state
<code>/login · /logout</code>	switch accounts

3 CLI Flags You'll Reach For

Flags by job

-p, --print "..."	print result & exit (no REPL)
-c, --continue	continue most recent session
-r, --resume [id]	pick / resume a session
--model <i>name</i>	opus · sonnet · haiku
--add-dir <i>path</i>	add another working dir
--allowedTools <i>list</i>	whitelist tools (comma-sep)
--disallowedTools <i>list</i>	blacklist tools
--permission-mode <i>m</i>	default · acceptEdits · plan · bypassPermissions
--output-format <i>fmt</i>	text · json · stream-json
--append-system-prompt	add to system prompt
--mcp-config <i>file.json</i>	load MCP servers from file
--session-id · --verbose · --debug	scripting & troubleshooting

One-liners for the win

```
# automate from the shell
claude -p "summarize git diff" | tee notes.md

# pipe stdin into Claude
cat error.log | claude -p "what failed?"

# JSON for downstream tools
claude -p "list todos" --output-format json

# plan mode (read-only, no edits)
claude --permission-mode plan

# YOLO mode in a sandbox / container
claude --dangerously-skip-permissions
```

Skip-permissions = root keys. Only use it inside a sandbox, container, or throwaway VM — Claude can run anything.

4 Where Claude Looks for Things

The Claude Code file map

~/.claude/ # personal, all projects | **settings.json** # your defaults | **CLAUDE.md** # personal memory ("you always TDD") | **commands/** # personal /slash commands → *.md | **agents/** # personal subagents → *.md | **plugins/** # installed plugins & marketplaces myrepo/.claude/ # shared with the team (commit it!) | **settings.json** # project-wide rules | **settings.local.json** # your overrides (.gitignore this) | **commands/** # project /slash commands | **agents/** # project subagents | **CLAUDE.md** # project context for everyone

Loading order: enterprise → user (~/.claude) → project (.claude) → local. Later layers override earlier ones.

CLAUDE.md — your memory

```
# Project: shop-api

## Stack
- Node 20, Postgres 16, Drizzle ORM
- Tests live in `tests/` (vitest)

## Conventions
- Always run `pnpm typecheck` after edits
- Never commit to `main` — open a PR

## Watch out for
- `legacy/` is read-only, skip it
```

Type # at the start of a message to append a note. Use /memory to edit, or /init to bootstrap.

settings.json — project rules

```
{
  "permissions": {
    "allow": ["Bash(npm test)", "Edit"],
    "deny": ["Bash(rm -rf:*)"],
    "defaultMode": "acceptEdits"
  },
  "model": "claude-sonnet-4-6",
  "env": { "CI": "1" },
  "hooks": { /* see §6 */ }
}
```

Allow patterns use a tool spec: Bash(npm:*), Edit(src/**), WebFetch(domain:docs.python.org).

5 Custom Commands & Subagents

/ Make your own slash command

```
# .claude/commands/review.md
---
description: Review my staged changes
argument-hint: [scope]
---
Run git diff --staged and review the
changes for bugs, missing tests, and naming.
Focus on $ARGUMENTS if given.
```

Now `/review` security just works — for everyone on the repo.

◇ Spawn a specialist subagent

```
# .claude/agents/test-runner.md
---
name: test-runner
description: PROACTIVELY runs & fixes
  failing tests after code changes.
tools: Bash, Read, Edit, Grep
model: sonnet
---
You are a focused test runner.
1. Run the tests.
2. Read failures.
3. Fix the smallest thing & re-run.
```

Type `/agents` to create / edit · Claude will dispatch them automatically when relevant.

6 Hooks · Run Code on Lifecycle Events

✕ Where each hook fires



Use them for: auto-format on save · block dangerous commands · log every tool call · post a Slack ping when Claude needs you · inject project state into context.

Block a tool with a hook

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{
        "type": "command",
        "command": "check-cmd.sh"
      }]
    }]
  }
}
```

Exit code 2 blocks the tool · stdout is fed back to Claude as feedback.

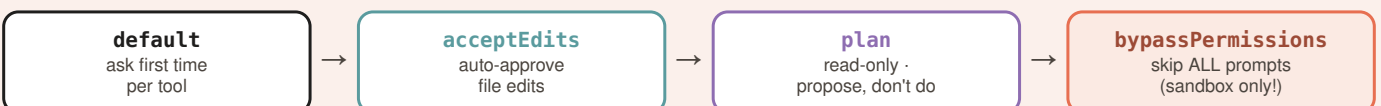
✓ Auto-format after every edit

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "prettier --write
\\$CLAUDE_FILE_PATHS\\"
      }]
    }]
  }
}
```

Type `/hooks` for a guided editor — no JSON wrangling required.

7 Permission Modes & Tool Safety

Cycle modes with **Shift+Tab**



Allow / deny rule recipes

```
{
  "permissions": {
    "allow": [
      "Read",
      "Edit(src/**)",
      "Bash(npm test)",
      "Bash(npm run *:*)",
      "WebFetch(domain:docs.anthropic.com)"
    ],
    "deny": [
      "Bash(rm -rf:*)",
      "Edit(secrets/**)",
      "Bash(curl:*)"
    ],
    "defaultMode": "default"
  }
}
```

Built-in tools at a glance

Read · Edit · Write	files in your repo
Bash	shell commands
Glob · Grep	find files & code
WebFetch · WebSearch	read & query the web
Agent (Task)	spawn subagent
TodoWrite	structured task list
NotebookEdit	edit Jupyter cells

Use /permissions for a guided UI to toggle each.

8 Plug In Tools with MCP

Add & manage servers

```
# add a stdio server (default)
claude mcp add github npx -y @modelcontextprotocol/
server-github

# pick the scope it lives in
claude mcp add -s project linear npx -y linear-mcp

# HTTP / SSE remote server
claude mcp add docs --transport http https://
api.docs.io/mcp

# list, inspect, remove
claude mcp list
claude mcp get github
claude mcp remove github
```

Scopes: local (just you, this dir) · project (commit to repo) · user (you, everywhere).

Use MCP tools in chat

/mcp	connect / auth servers (OAuth)
@server:resource	attach an MCP resource
/mcp__srv__prompt	run a prompt exposed by a server
--mcp-config f	load servers from JSON file

Popular servers: github · filesystem · postgres · puppeteer · slack · linear · sentry. Browse the [MCP registry](#) for more.

9 Plugins & Marketplaces

Plugin commands

/plugin	interactive plugin manager
/plugin install name	install from a marketplace
/plugin list	show installed
/plugin uninstall name	remove
/plugin marketplace add url	subscribe to a marketplace

A plugin can bundle **commands** · **agents** · **hooks** · **MCP servers** in one install.

Power-user tricks

- ▶ Drag-and-drop a file into the terminal to @-reference it.
- ▶ Pipe a screenshot URL or paste an image — Claude can see it.
- ▶ Add output-style in ~/.claude/output-styles/ for custom personas.
- ▶ Use a statusline script to show git branch, model, cost in the prompt.
- ▶ claude config set --global preferredNotifChannel iterm2 for desktop pings.
- ▶ Run inside tmux — sessions survive disconnects.
- ▶ Wrap risky tasks in a Docker container + --dangerously-skip-permissions.